

Arrays

Mahir Labib Dihan
Lecturer, CSE, BUET



Table of Contents

1. Introduction to Arrays
2. Working with Arrays
3. Some Array Problems
4. Searching Algorithms
5. Selection Sort

Introduction to Arrays



The Problem: Storing Multiple Values

Scenario: Read 10 numbers from the keyboard and store them

```
// solution #1  
int a, b, c, d, e, f, g, h, i, j;  
  
printf("Enter a number: ");  
scanf("%d", &a);  
  
printf("Enter a number: ");  
scanf("%d", &b);  
  
// ...  
printf("Enter a number: ");  
scanf("%d", &j);
```

Many Variables

Many lines of code

The Problem: Storing Multiple Values

Scenario: Read ¹⁰⁰~~10~~ numbers from the keyboard and store them

```
// solution #1
int a, b, c, d, e, f, g, h, i, j;

printf("Enter a number: ");
scanf("%d", &a);

printf("Enter a number: ");
scanf("%d", &b);

// ...
printf("Enter a number: ");
scanf("%d", &j);
```

Many Variables

Many lines of code

What is an Array?

An array is an ordered list of values of similar type

The entire array
has a single name


a

Each value has a numeric index



0	1	2	3	4	5	6	7	8	9
79	87	94	82	67	98	87	81	74	91

An array of size N is indexed from zero to N-1

This array holds 10 values that are indexed from 0 to 9

Arrays: Example

An array with 8 elements of type double

```
double x[8];
```

Array x

x[0]	x[1]	x[2]	x[3]	x[4]	x[5]	x[6]	x[7]
16.0	12.0	6.3	8.2	2.5	12.4	14.0	-3.6

```
printf("%lf", x[3]); // 8.2
```

Arrays: Key Properties

- The values held in an array are called *array elements*
- An array stores multiple values of the same type
- Therefore, we can create an array of ints, an array of floats, an array of doubles , an array of chars.
- Array size is determined at declaration
- **Note:** First element is at index 0, not 1

Working with Arrays



Array Declaration

Syntax

```
data_type array_name[size];
```

Examples:

```
int marks[50]; // 50 integers
float temperatures[7]; // 7 floating - point numbers
char name[20]; // 20 characters (a string)
double prices[100]; // 100 double - precision numbers
```

Important Notes:

- Size must be a positive integer constant (or constant expression; no variables)
- Arrays declared inside functions have garbage values initially

How to assign values?

Initialize at Declaration

```
int arr[5] = {10 , 20, 30, 40, 50}; // All 5 values specified
```

Partial Initialization

```
int arr[5] = {10 , 20}; // arr = {10 , 20, 0, 0, 0}  
                        // Remaining elements set to 0
```

Initialize All to Zero

```
int arr[5] = {0}; // All elements are 0
```

Let Compiler Determine Size

```
int arr[] = {1, 2, 3, 4, 5}; // Size automatically set to 5
```

How to assign values?

Use assignment operator

```
int a[6];  
a[0] = 3;  
a[1] = 6;
```

Assign Using Loop

```
int a[6];  
for (i = 0; i < 6; i++) {  
    a[i] = i * 10; // arr = {0, 10, 20, 30, 40}  
}
```

Array index could be constant, integer variable or expressions that generate integers

C does not check array bounds! Accessing `a[6]` or `a[-1]` causes undefined behavior (crash, garbage, or silent corruption).

How to assign values?

Use scanf to input in the array:

```
int a[6];  
scanf("%d", &a[0]);  
scanf("%d", &a[1]);  
...  
scanf("%d", &a[5]);
```

Assign Using Loop

```
int a[6];  
for (i = 0; i < 6; i++) {  
    scanf("%d", &a[i]);  
}
```

Overview of Bracket Types

Bracket	Technical Name	Primary Purpose in C
()	Parentheses	Functions, control flow conditions, and math precedence
{ }	Curly Braces	Code blocks, variable scope, and structure initialization
[]	Square Brackets	Array declaration and indexing
< >	Angle Brackets	Header file inclusion via preprocessor

Some Array Problems



Example: Sum and Average

Task: Calculate sum and average of array elements.

```
int arr[5] = {10 , 20, 30, 40, 50};
int n = 5;

int sum = 0;
for (int i = 0; i < n; i++) {
    sum += arr[i];
}

float average = (float) sum / n;

printf("Sum = %d\n", sum); // Sum = 150
printf("Average = %.2f\n", average); // Average = 30.00
```

Example: Sum and Average

Task: Calculate sum and average of n array elements. Take n and array elements as input.

```
int n, i;  
scanf("%d", &n);  
int arr[n];  
for (i = 0; i < n; i++) {  
    scanf("%d", &arr[i]);  
}
```

What's wrong here?

Can't declare variable length array like: `int arr[n];`

Example: Sum and Average

Task: Calculate sum and average of n array elements. Take n and array elements as input.

where $0 < n \leq 100$

```
int n, i;
int arr[100];
scanf("%d", &n);
for (i = 0; i < n; i++) {
    scanf("%d", &arr[i]);
}
```

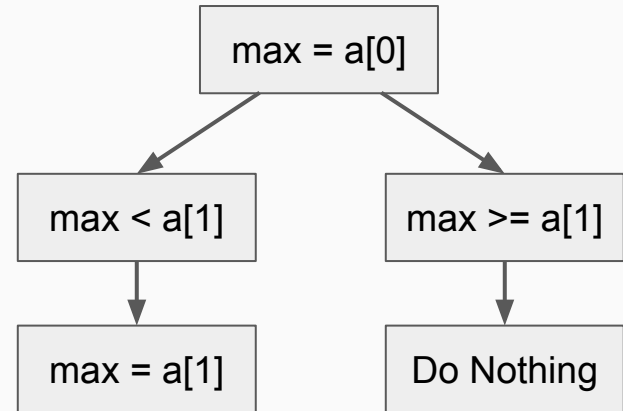
Exercise

1. Suppose an array has N students marks. Find grade of each student.
2. Take N numbers as input and store them in an array. Print all odd numbers in the array.

Find Maximum of Array

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]	a[10]	a[11]
4	40	30	70	60	70	88	99	10	87	91	65

Initially assume first element is the maximum



Find Maximum of Array

Task: Find the maximum number in an array of integers.

```
#include <stdio.h>

int main()
{
    int a[12] = {4, 40, 30, 70, 60, 70, 88, 99, 10, 87, 91, 65};
    int i;
    // Find The Maximum Element
    int max = a[0]; // pick the first number as the current maximum
    for (i = 1; i < N; i++)
    {
        if (max < a[i])
            max = a[i];
    }
    printf("The maximum value in the array is %d.\n", max);
}
```

Find Maximum of Array and Index

Task: Find the maximum number and its index in an array of integers.

```
int main()
{
    int a[12] = {4, 40, 30, 70, 60, 70, 88, 99, 10, 87, 91, 65};
    int i;
    // Find The Maximum Element and its index
    int max = a[0];    // initial guess: a[0] is the maximum value
    int idx = 0;       // initial guess: the maximum value is at index 0
    for (i = 1; i < N; i++)
    {
        if (max < a[i]) {
            max = a[i];
            idx = i;
        }
    }
    printf("The maximum value in the array is %d.\n", max);
    printf("It is located at index: %d\n", idx);
}
```

Largest and Second largest

Task: Print largest and second largest element of an array (which has at least two elements).

```
if (a[0] > a[1]){
    largest = a[0];
    secL = a[1];
}
else{
    largest = a[1];
    secL = a[0];
}
for (i = 2; i < N; i++) {
    t = a[i];
    if (t >= largest){
        secL = largest;
        largest = t;
    }
    else if (t > secL) secL = t;
}
printf("The largest: %d second largest: %d", largest, secL)
```

Distinct Elements in Sorted Array

Task: Count the number of distinct (unique) elements in a sorted (ascending) array.

1	1	2	2	2	3	4	4
---	---	---	---	---	---	---	---

Answer: 4 distinct elements (1, 2, 3, 4)

Distinct Elements in Sorted Array

Task: Count the number of distinct (unique) elements in a sorted (ascending) array.

1	1	2	2	2	3	4	4
---	---	---	---	---	---	---	---

Green = first occurrence (distinct), **Gray** = duplicate

Answer: 4 distinct elements (1, 2, 3, 4)

Algorithm:

```
count ← 1
for i ← 1 to n-1 do
    if arr[i] ≠ arr[i-1] then
        count ← count + 1
return count
```

Key Insight:

In a sorted array, duplicates are always adjacent.

Count when current ≠ previous.

Distinct Elements in Sorted Array

Task: Count the number of distinct (unique) elements in a sorted (ascending) array.

```
#include <stdio.h>

int main () {
    int arr[] = {1, 1, 2, 2, 2, 3, 4, 4};
    int n = 8;
    int count = 1; // First element is always distinct
    for (int i = 1; i < n; i++) {
        if (arr[i] != arr[i - 1]) {
            count++;
        }
    }
    printf ("Distinct elements : %d\n", count);
    // Output: 4
    return 0;
}
```

Distinct Elements in Unsorted Array

Task: Count distinct elements when array is not sorted.

4	6	2	9	7	7	4	2
---	---	---	---	---	---	---	---

Approach: For each element, check if it appeared before. If not, it's the first occurrence and we can increase count.

Distinct Elements in Unsorted Array

Task: Count distinct elements when array is not sorted.

Approach: For each element, check if it appeared before.

```
int count = 0;
for (int i = 0; i < n; i++) {
    // Check if arr[i] appeared before index i
    int isDuplicate = 0;
    for (int j = 0; j < i; j++) {
        if (arr[j] == arr[i]) {
            isDuplicate = 1;
            break;
        }
    }
    if (!isDuplicate) count++;
}
// arr = {5, 2, 8, 2, 5, 1} -> Distinct : 4 (5, 2, 8, 1)
```

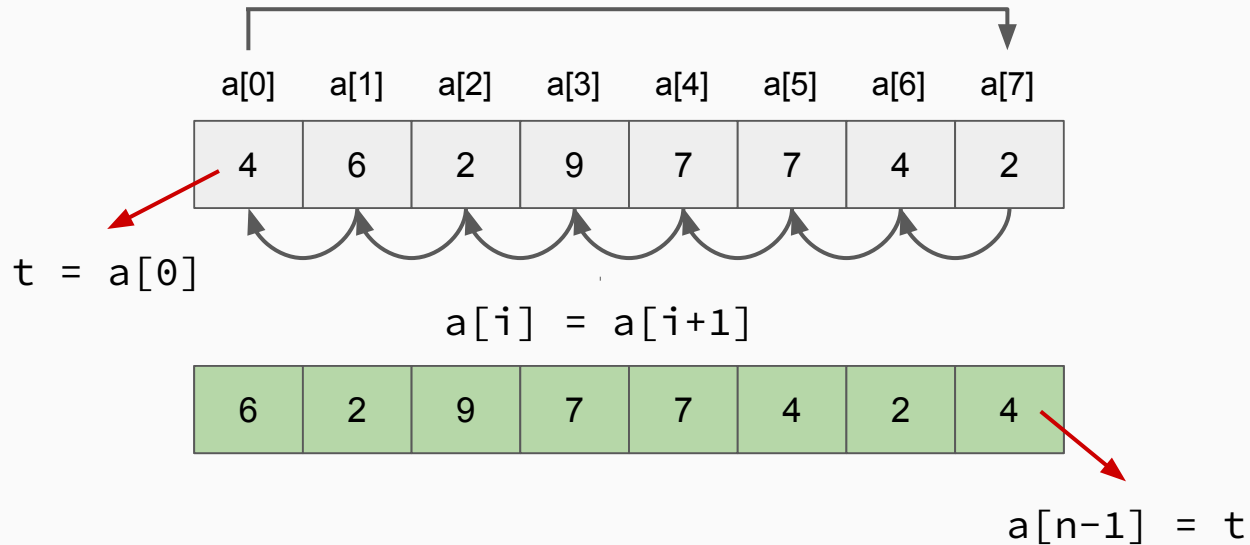
Left Rotate Array: Concept

Task: Rotate all elements to the left by one position.



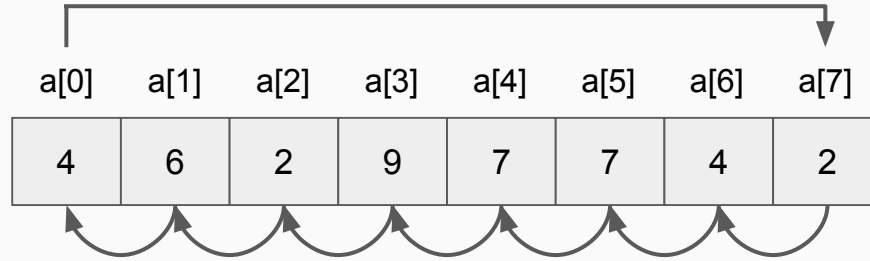
Left Rotate Array: Concept

Task: Rotate all elements to the left by one position.



Left Rotate Array: Concept

Task: Rotate all elements to the left by one position.



1. Save the first element
2. Shift all elements one position to the left
3. Put the saved element at the last position

Left Rotate Array: Solution

```
int arr[8] = {4, 6, 2, 9, 7, 7, 4, 2};
int n = 8, i;
int t = arr[0]; // Save first element
// Shift all elements left by 1
for (i = 0; i < n - 1; i++) {
    arr[i] = arr[i + 1];
}
arr[n - 1] = first; // Put first at end

printf("Array elements after left rotation...\n")
for (i = 0; i < n; i++) {
    printf ("%d ", arr[i]);
}
```

Searching Algorithms



Why Search Algorithms?

Common Problem: Given an array and a target value, does the value exist? Where?

Real-world Examples:

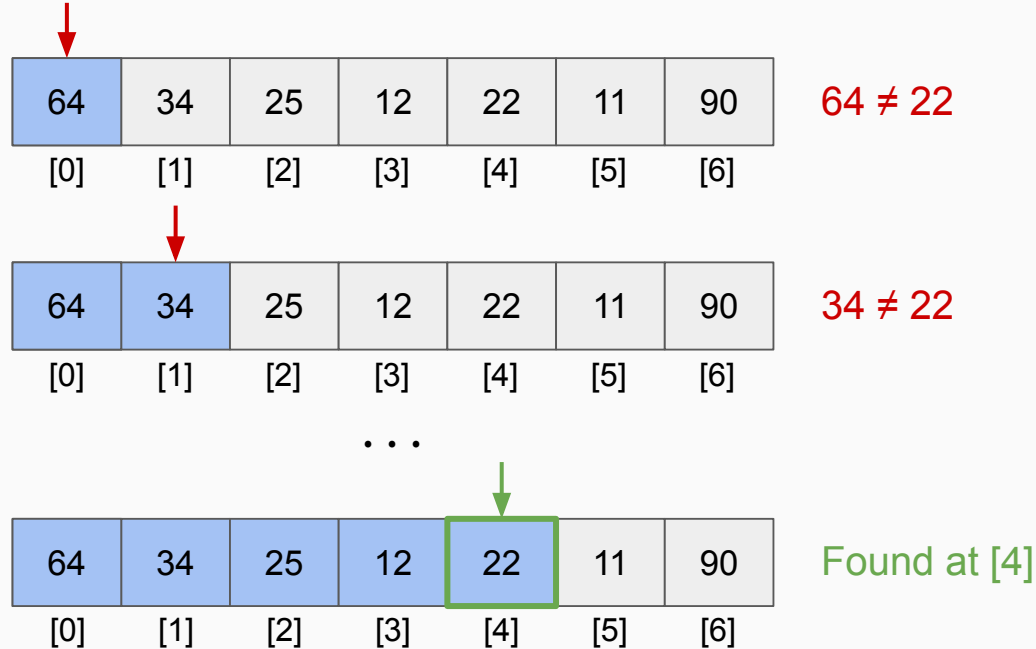
- Finding a contact in your phone
- Looking up a word in a dictionary
- Checking if an item is in stock
- Finding a student's record by ID

Two Main Approaches:

1. **Linear Search:** Check each element one by one
2. **Binary Search:** Divide and conquer (requires sorted array)

Example: Successful Linear Search

Searching for 22 in array:



Example: Failed Linear Search

Searching for 14 in array:



64	34	25	12	22	11	90
[0]	[1]	[2]	[3]	[4]	[5]	[6]

$64 \neq 14$



64	34	25	12	22	11	90
[0]	[1]	[2]	[3]	[4]	[5]	[6]

$34 \neq 14$

...



64	34	25	12	22	11	90
[0]	[1]	[2]	[3]	[4]	[5]	[6]

$90 \neq 14$

Linear Search: Unsorted

Task: Find the index of a number in an unsorted array of integers.

```
int a[12] = {4, 21, 36, 14, 62, 91, 8, 22, 7, 81, 77, 10};
int target = 62;
int i, idx = -1;
for (i=0; i< N; i++)
{
    if (a[i] == target)
    {
        idx = i;
        break;
    }
}
if (idx == -1)
    printf("Target not found.\n");
else
    printf("Target found at index: %d\n", idx);
```

Linear Search

Pros:

- The most basic
- Very easy to implement
- The array DOESN'T have to be sorted

Cons:

- All array elements must be visited if the search fails
- Could be very slow

Linear Search: Sorted

Task: Find the index of a number in an **sorted** array of integers.

Target = 14



11	12	22	25	34	64	90
[0]	[1]	[2]	[3]	[4]	[5]	[6]

11 ≠ 14



11	12	22	25	34	64	90
[0]	[1]	[2]	[3]	[4]	[5]	[6]

12 ≠ 14



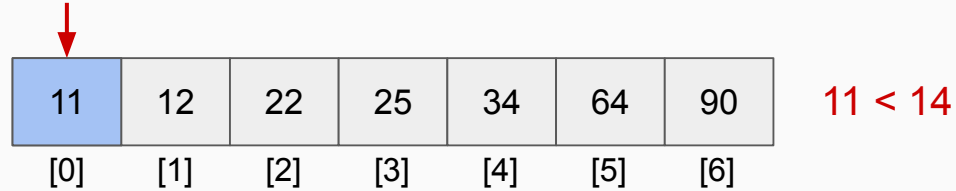
11	12	22	25	34	64	90
[0]	[1]	[2]	[3]	[4]	[5]	[6]

22 ≠ 14

Linear Search: Sorted

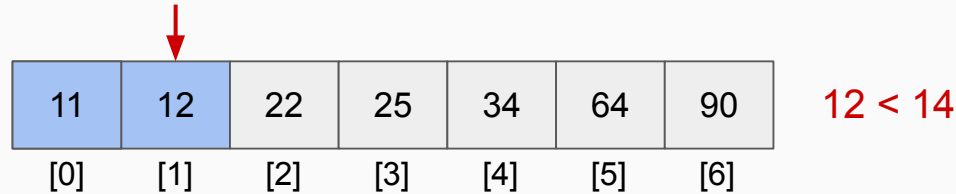
Task: Find the index of a number in an **sorted** array of integers.

Target = 14



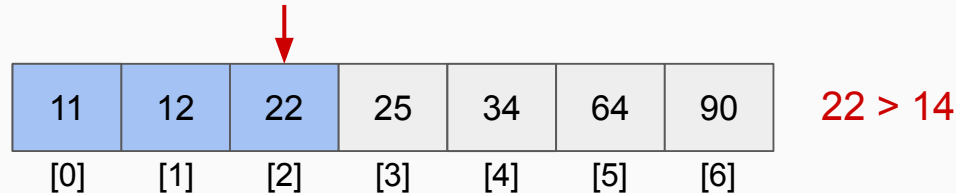
11	12	22	25	34	64	90
[0]	[1]	[2]	[3]	[4]	[5]	[6]

$11 < 14$



11	12	22	25	34	64	90
[0]	[1]	[2]	[3]	[4]	[5]	[6]

$12 < 14$



11	12	22	25	34	64	90
[0]	[1]	[2]	[3]	[4]	[5]	[6]

$22 > 14$

Linear Search: Sorted

Task: Find the index of a number in an **sorted** array of integers.

```
int a[12] = {4, 21, 36, 14, 62, 91, 8, 22, 7, 81, 77, 10};
int target = 62;
int i, idx = -1;
for (i=0; i< N; i++)
{
    if (a[i] == target)
    {
        idx = i;
        break;
    }
    else if (a[i] > target)
        break;
}
if (idx == -1) printf("Target not found.\n");
else printf("Target found at index: %d\n", idx);
```

Linear Search: Analysis

- If the list is unsorted we have to search all numbers before we declare that the target is not present in the array.
- Because the list is sorted we can stop as soon as we reach a number that is greater than our target.
- Can we do even better?

The Guessing Game

Let's play a game:

- I am thinking of a number between **1** and **1000**.
- You guess a number.
- I will tell you if my number is **Higher**, **Lower**, or **Equal**.

Strategy 1 (Linear): Guess 1, 2, 3, 4...

- Worst case: 1000 guesses! (Too slow)

Strategy 2 (Binary): Guess 500 (Middle).

- If I say “Lower”, the answer is 1-499. (We eliminated 500 numbers!)
- Next guess: 250.
- If I say “Higher”, the answer is 251-499.
- Max guesses needed: ≈ 10 ($2^{10} = 1024$)
- This is the power of Binary Search!

Binary Search: Concept

Prerequisite: Array must be sorted!

Idea: Divide the search space in half at each step.

1. Compare target with middle element
2. If equal, found!
3. If target < middle, search left half
4. If target > middle, search right half
5. Repeat until found or search space is empty

Analogy: Finding a word in a dictionary

- Open to the middle
- Is the word before or after?
- Go to the appropriate half and repeat

Binary Search: Step-by-Step

Searching for 34 in array:



7	11	12	22	25	34	64	90	95
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

$34 > 25 \Rightarrow$ Go right



7	11	12	22	25	34	64	90	95
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

$34 < 64 \Rightarrow$ Go left



7	11	12	22	25	34	64	90	95
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

Found at [5]

Binary Search: Step-by-Step

Searching for 34 in array:

$M = (L+R) / 2$

L		M		R						
↓	7	11	12	22	25	34	64	90	95	↓
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	

$34 > 25 \Rightarrow$ Go right

$L = M+1$

L		M		R						
	7	11	12	22	25	34	64	90	95	↓
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	

$34 < 64 \Rightarrow$ Go left

$R = M-1$

L		M		R						
↓	7	11	12	22	25	34	64	90	95	↓
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	

Found at [5]

Binary Search: Code

```
int idx = -1, l = 0, r = N-1;
int m = (l+r)/2;

while (r >= l) {
    if( a[m] == target)
    {
        idx = m; // Found it!
        break;
    }
    else if(a[m] > target)
        l = m - 1;
    else
        r = m + 1;

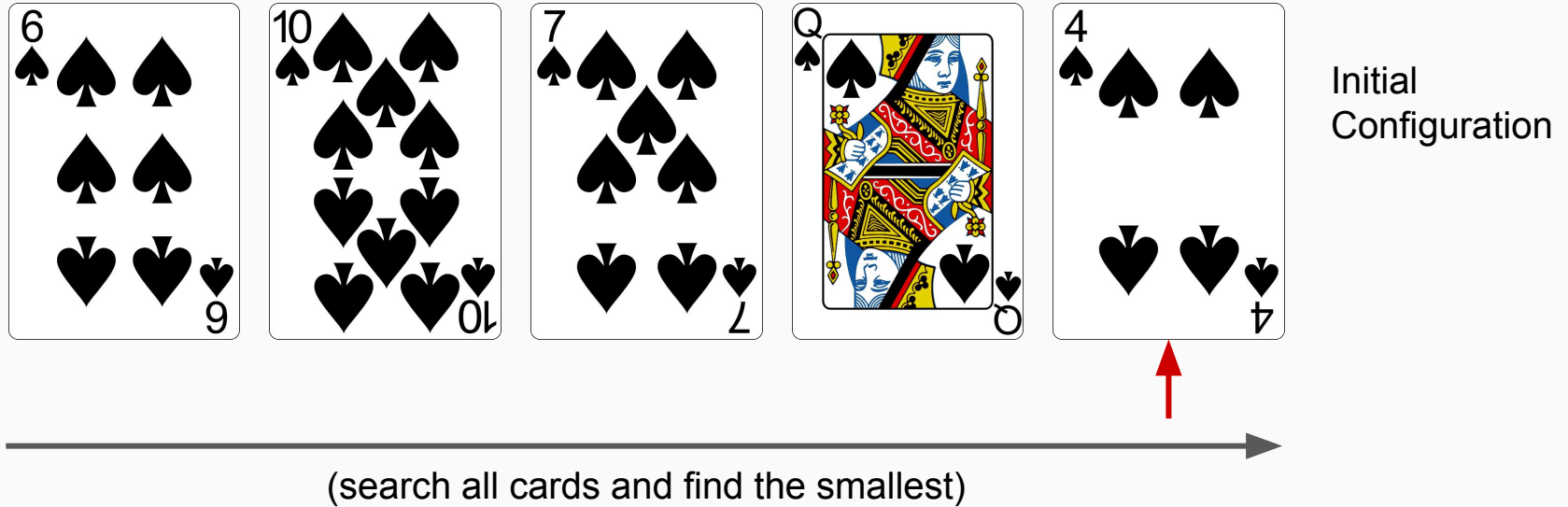
    m = (l + r)/2; // integer division!
}

if(idx == -1)
    printf("Target not found.\n");
else
    printf("Target found at index: %d\n", idx);
```

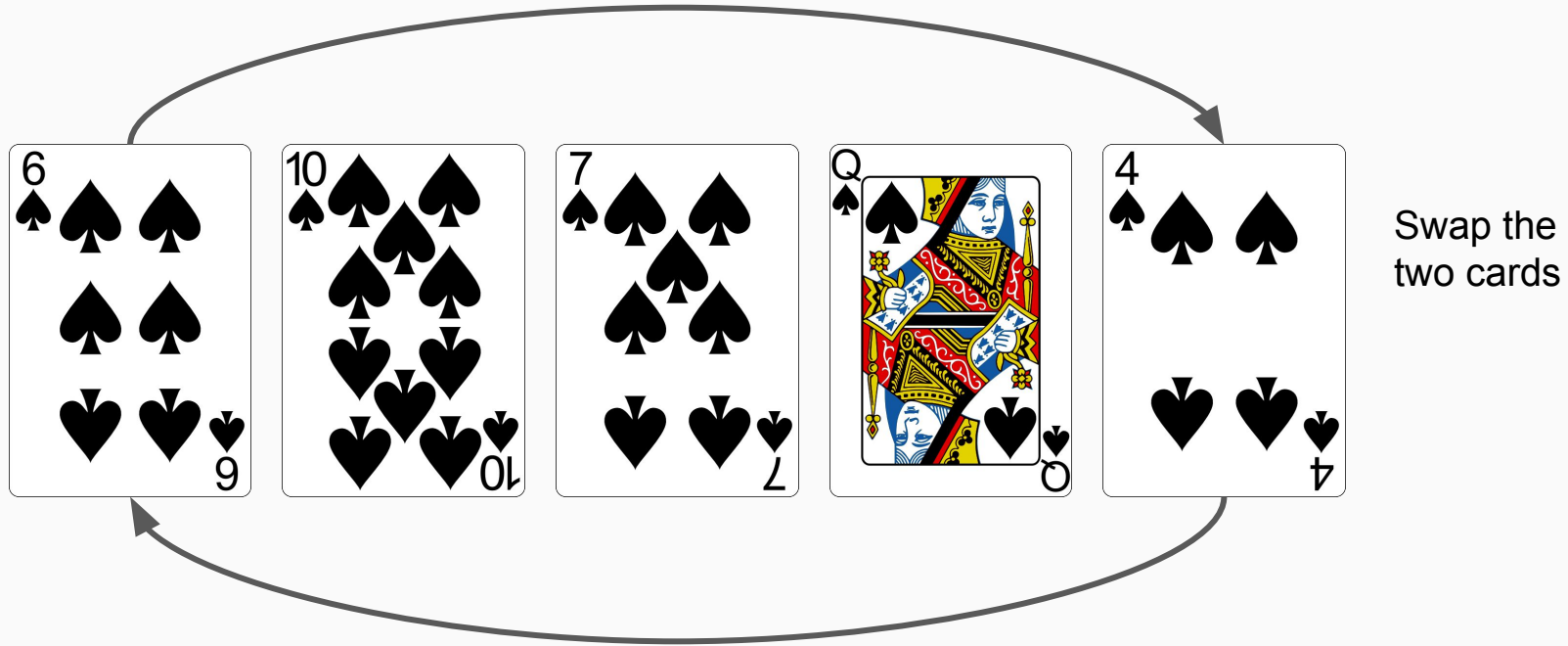
Selection Sort



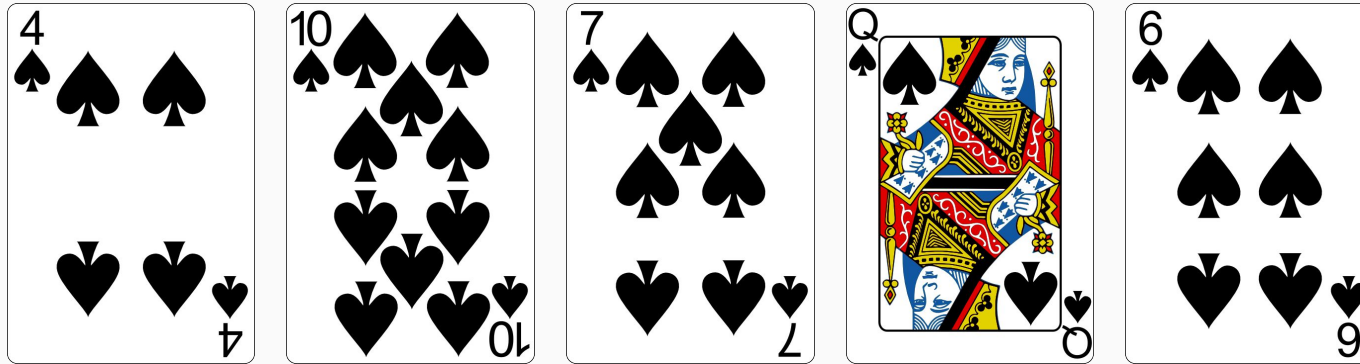
Selection Sort: Visualization



Selection Sort: Visualization

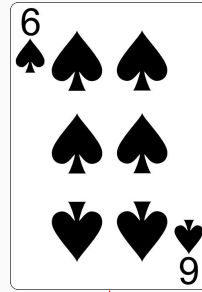
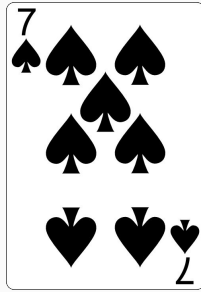
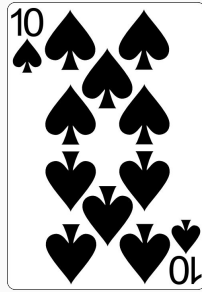
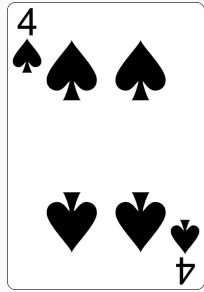


Selection Sort: Visualization



Swap the
two cards

Selection Sort: Visualization



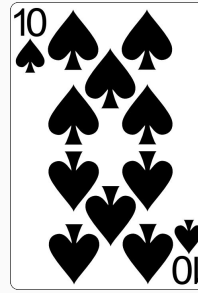
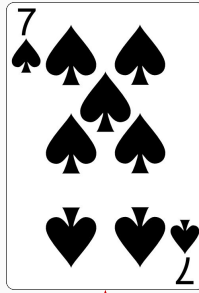
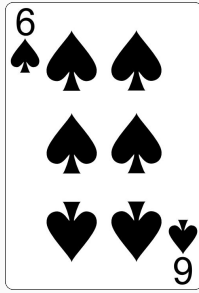
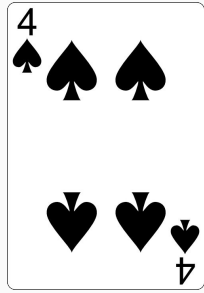
Among the remaining cards, 6 is the smallest



Sorted

Unsorted

Selection Sort: Visualization

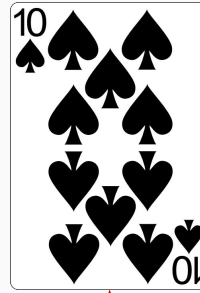
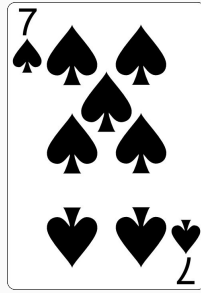
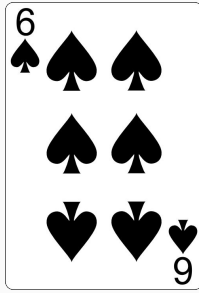
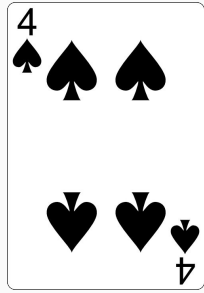


Among the remaining cards, 7 is the smallest

Sorted

Unsorted

Selection Sort: Visualization



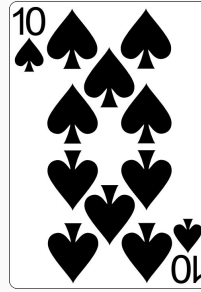
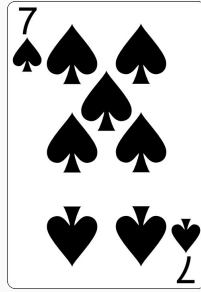
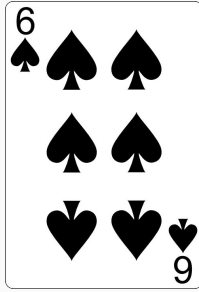
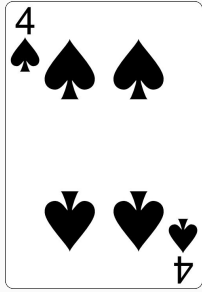
Among the remaining cards, 10 is the smallest



Sorted

Unsorted

Selection Sort: Visualization

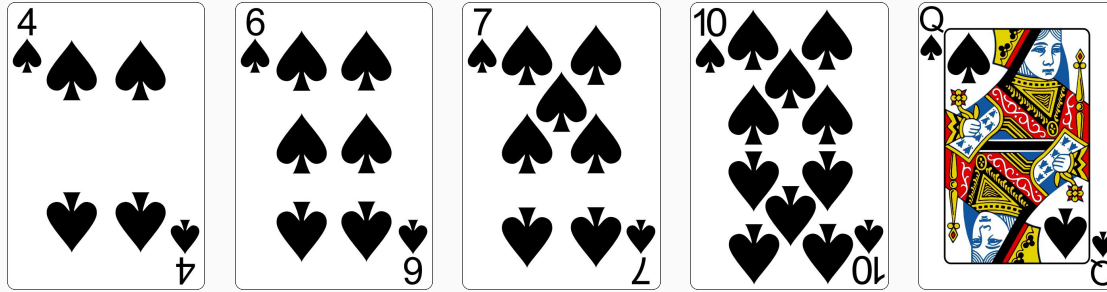


Sorted

Unsorted

We are down to the last card. Because there is only one and Because we know that it is Smaller than all the rest We don't need to do anything Else with it. This is why the Algorithm goes up to $< N-1$

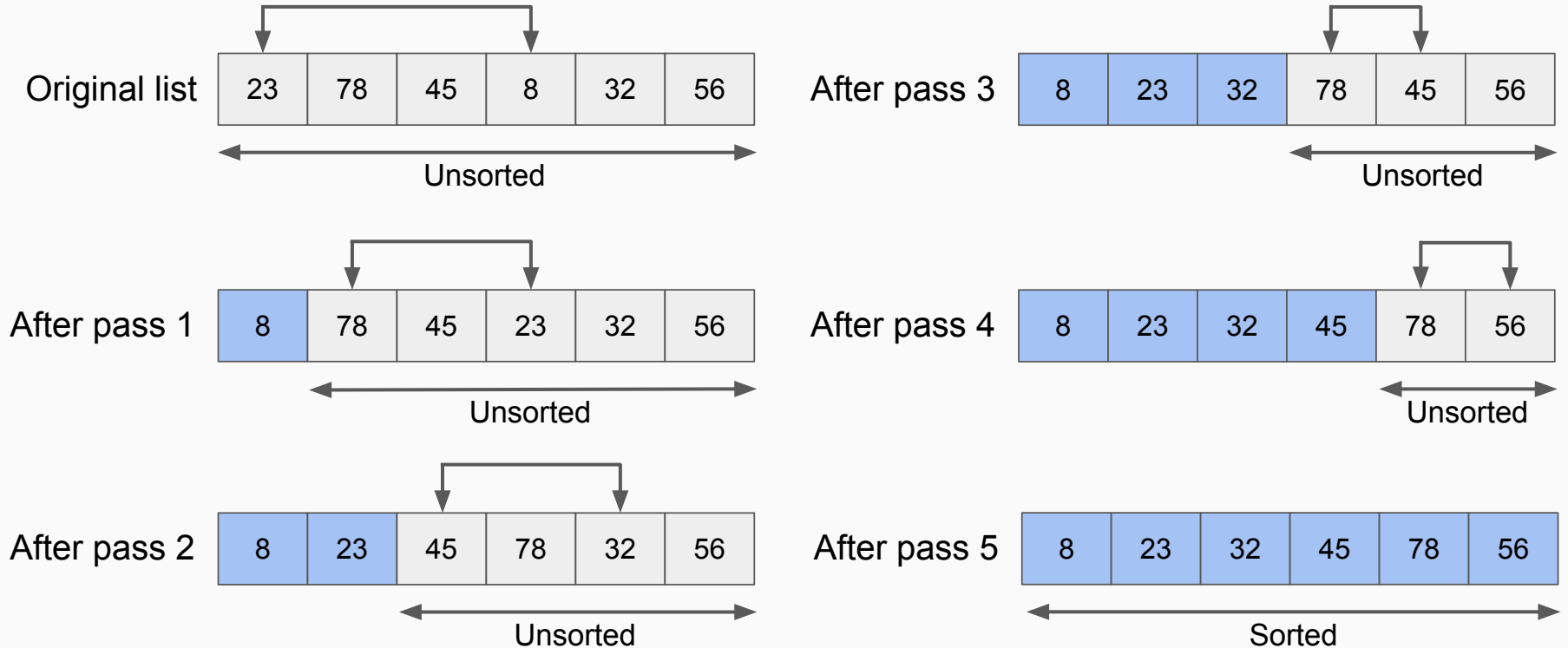
Selection Sort: Visualization



All cards are now sorted.

Sorted

Selection Sort: Example



Selection Sort: Code

```
int i, j, tmp;
// Sort the array using Selection Sort
int minIdx;
for (i = 0; i < N-1; i++)
{
    minIdx = i;
    for(j = i+1; j < N; j++) {
        if (a[j] < a[minIdx]) {
            minIdx = j;
        }
    }
    tmp = a[i];
    a[i] = a[minIdx];
    a[minIdx] = tmp;
}
```

Acknowledgement

- Dr. Ashikur Rahman, Professor, CSE, BUET
- Mohammad Sadat Hossain, Lecturer, CSE, BUET